

MODELAGEM E PERSISTÊNCIA DE DADOS COM ORM

Sumário:

1. Introdução ao Object-Relational Mapping.....	2
1.1 O que é o ORM e qual problema ele resolve?.....	2
1.2 Principais ORMs no ecossistema Node.js.....	2
1.3 Comparação entre ORM e SQL manual.....	3
1.4 Como o ORM realiza o “mapeamento”	6
1.5 Benefícios do uso de ORM.....	7
1.6 Desvantagens do ORM.....	8
1.7 Boas práticas de integração entre as abordagens.....	8
2. Instalação e configuração do Prisma.....	9
2.1 O que é o Prisma ORM.....	9
2.2 Etapas de instalação e configuração.....	9
3 Operações CRUD com Prisma Client.....	16
3.1 Definindo o Modelo.....	16
3.2 Criando o Prisma Client.....	17
3.3 CREATE – Inserindo um novo registro.....	17
3.4 READ – Consultando dados.....	19
3.5 UPDATE – Atualizando registros existentes.....	21
3.6 DELETE – Excluindo registros.....	22
4. Atualização de modelos, migrações e seed de dados.....	24
4.1 Modelagem no schema.prisma.....	24
4.2 Migrações: criar, versionar e aplicar.....	25
4.3 Seed de dados.....	28
5 Relações (1:1, 1:N, N:N) em nível de código.....	34
5.1 Relação 1:1 (User ↔ Profile).....	34
5.2 Relação 1:N (User → Post).....	36
5.3 Relação N:N (Post ↔ Categoria).....	39

1. Introdução ao Object-Relational Mapping

1.1 O que é o ORM e qual problema ele resolve?

No desenvolvimento de aplicações web modernas, é comum que os dados sejam armazenados em bancos de dados relacionais, como **MySQL**, **PostgreSQL** ou **SQLite**, onde as informações são organizadas em **tabelas** compostas por colunas e linhas. Já no código JavaScript, trabalhamos com **objetos**, que representam entidades e seus comportamentos. Essa diferença estrutural cria o que se chama de *impedância objeto-relacional*, ou seja, a dificuldade em traduzir objetos de código em registros de banco de dados e vice-versa (FOWLER, 2002).

O **ORM (Object-Relational Mapping)**, ou **Mapeamento Objeto-Relacional**, é uma técnica que busca resolver esse problema, permitindo que o programador interaja com o banco de dados utilizando **objetos e métodos** em vez de **consultas SQL diretas** (WENDE, 2019). Em outras palavras, o ORM atua como uma camada intermediária entre o código e o banco de dados, automatizando o processo de conversão de dados.

Por meio dessa abstração, o desenvolvedor não precisa se preocupar em montar consultas SQL detalhadas. Basta manipular objetos no código, e o ORM se encarrega de gerar, executar e interpretar os comandos SQL adequados (POWELL, 2020). Porém, é importante compreender que o ORM **não substitui o conhecimento de SQL**. Ele apenas simplifica a interação com o banco, mas por trás de cada método (**findMany**, **create**, **update**, etc.) ainda existe uma consulta SQL sendo executada (EVANS, 2004).

1.2 Principais ORMs no ecossistema Node.js

Existem diversos ORMs disponíveis para Node.js, cada um com suas características, vantagens e limitações. Entre os mais populares estão o **Sequelize**, o **Prisma** e o **TypeORM**.

1.2.1 Sequelize

O **Sequelize** é um dos mais antigos e amplamente utilizados. Ele segue o padrão *Data Mapper* e permite a definição de modelos em JavaScript ou TypeScript, além de oferecer suporte a várias bases de dados (MySQL, PostgreSQL, SQLite e MariaDB). Sua principal

vantagem é a maturidade e o amplo suporte da comunidade, embora sua sintaxe seja considerada mais verbosa e sua integração com TypeScript menos intuitiva (Sequelize, 2025).

1.2.2 Prisma

O **Prisma ORM**, por outro lado, representa uma nova geração de ferramentas de mapeamento objeto-relacional. Diferentemente do Sequelize, o Prisma é baseado em um **arquivo de esquema declarativo** (`schema.prisma`), no qual o desenvolvedor define os modelos e suas relações. A partir desse arquivo, o Prisma gera automaticamente um **cliente tipado** (`@prisma/client`), que fornece uma API de alto nível para realizar operações no banco de dados. Essa abordagem proporciona uma experiência de desenvolvimento mais produtiva, segura e coerente, especialmente em projetos que utilizam TypeScript (Prisma, 2024). Por exemplo, para listar usuários, basta escrever `await prisma.user.findMany()`, sem se preocupar com a estrutura da consulta SQL subjacente.

1.2.3 TypeORM

Outro ORM amplamente utilizado é o **TypeORM**, que se destaca pela integração profunda com o TypeScript e o uso de **decorators** (anotações) para definir as entidades e seus relacionamentos. Ele é bastante popular em projetos que seguem padrões de arquitetura orientada a classes, permitindo a escolha entre os padrões *Active Record* e *Data Mapper*. Sua curva de aprendizado é um pouco mais elevada, mas oferece grande flexibilidade e poder de personalização (TypeORM, 2025).

1.3 Comparação entre ORM e SQL manual

1.3.1 Visão Geral

Ao desenvolver uma aplicação que se conecta a um banco de dados, há duas formas principais de realizar as operações de persistência:

1. Escrever **consultas SQL manuais**, interagindo diretamente com o banco;
2. Utilizar um **ORM (Object-Relational Mapping)**, que abstrai o SQL e oferece uma interface orientada a objetos.

Ambas as abordagens são válidas e amplamente utilizadas. A escolha entre uma e outra depende de fatores como **complexidade da aplicação**, **nível de abstração desejado**, **desempenho esperado** e **experiência da equipe** (FOWLER, 2002; POWELL, 2020).

De acordo com Powell (2020), o ORM funciona como um *“tradutor entre dois mundos”*: o mundo relacional (bancos de dados) e o mundo orientado a objetos (código JavaScript/TypeScript). Essa tradução simplifica o desenvolvimento, mas pode esconder parte da complexidade do banco. Já o uso de SQL manual oferece maior controle, ao custo de mais código e maior risco de erros humanos.

1.3.2 SQL manual: controle total, porém mais verboso

No **SQL manual**, o desenvolvedor escreve as instruções diretamente, utilizando bibliotecas como **mysql2**, **pg** ou **sqlite3** para se conectar e executar consultas. Isso garante **controle total sobre o desempenho**, as **otimizações** e os **detalhes da consulta**, mas exige um bom domínio da linguagem SQL e das particularidades do banco.

Vejamos abaixo o uso de SQL manual com Node.js:

```
import mysql from "mysql2/promise";

// Cria a conexão
const connection = await mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "1234",
  database: "empresa"
});

// Consulta SQL manual
const [rows] = await connection.execute(
  "SELECT id, nome, email FROM users WHERE email = ?",
  ["ana@ifrs.edu.br"]
);

console.log(rows[0]);
```

Neste exemplo:

- O programador é responsável por escrever o SQL e garantir sua segurança.

- É preciso escapar manualmente os parâmetros para evitar **SQL Injection** (OWASP, 2023).
- O resultado vem como uma matriz ou objeto genérico e precisa ser convertido em estruturas de dados apropriadas.
- Alterações no banco (ex.: mudar o nome de uma coluna) exigem ajustes manuais em todas as consultas.

Segundo Wenderlich (2021), esse tipo de abordagem oferece “*transparência total sobre o que está acontecendo*”, o que pode ser benéfico em sistemas de alta performance ou quando há necessidade de consultas extremamente otimizadas.

1.3.3 ORM: abstração e produtividade

Com o uso de um **ORM**, o código se torna mais expressivo, reduzindo a quantidade de comandos SQL explícitos. Em vez de escrever consultas, o desenvolvedor utiliza **métodos** e **modelos** definidos em JavaScript para manipular os dados. O ORM converte essas operações em instruções SQL adequadas para o banco configurado (PRISMA, 2024).

Venhamos um exemplo de uso do ORM Prisma:

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

// Consulta equivalente ao SELECT anterior
const user = await prisma.user.findUnique({
  where: { email: "ana@ifrs.edu.br" },
  select: { id: true, nome: true, email: true }
});

console.log(user);
```

Neste exemplo:

- O ORM Prisma gera o SQL automaticamente.
- O código fica mais legível e fácil de manter.
- Há proteção automática contra SQL Injection e validação de tipos.
- Mudanças na estrutura do banco (por exemplo, adição de uma coluna) podem ser controladas por **migrations**, sem alterar manualmente o código das rotas.

Segundo Martins e Pereira (2022), “os ORMs aumentam a produtividade do desenvolvedor ao permitir que o acesso ao banco seja tratado como parte da lógica de negócio, e não como uma camada técnica isolada”.

1.4 Como o ORM realiza o “mapeamento”

De acordo com Wenderlich (2021), o processo de mapeamento objeto-relacional consiste em associar cada **entidade do código** a uma **tabela do banco de dados**. Assim:

Conceito no código	Equivalente no banco de dados
Classe User	Tabela users
Atributo email	Coluna email
Objeto new User()	Linha (registro)

Esse mapeamento permite que o desenvolvedor manipule objetos em memória e o ORM traduza automaticamente essas ações em comandos SQL equivalentes. Por exemplo, ao criar um novo usuário, o ORM executa internamente o comando **INSERT INTO users (...) VALUES (...)**, sem que o programador precise escrever essa consulta manualmente (POWELL, 2020).

1.4.1 Exemplo simples de modelo com ORM

No Prisma, o modelo de dados é definido em um arquivo chamado **schema.prisma**:

```
model User {  
  id      Int      @id @default(autoincrement())  
  nome    String  
  email   String   @unique  
}
```

Esse modelo representa uma tabela **User** com três colunas: **id**, **nome** e **email**. Ao executar o comando **npx prisma migrate dev --name init**, o ORM cria a tabela automaticamente no banco.

No código JavaScript, a inserção de um novo registro é feita assim:

```
const novoUsuario = await prisma.user.create({  
  data: {  
    nome: "Ana Souza",  
    email: "ana@ifrs.edu.br"  
  }  
});
```

Nesse processo, o Prisma gera internamente um comando **INSERT INTO**, executa a operação e retorna o objeto recém-criado. O programador interage apenas com **métodos e objetos**, sem precisar conhecer a sintaxe SQL.

1.4.2 O que o ORM faz nos bastidores

Quando um método como **prisma.user.create()** é executado, o ORM realiza automaticamente várias etapas (PRISMA, 2024):

1. Abre a conexão com o banco.
2. Valida os dados de entrada.
3. Gera a consulta SQL correspondente.
4. Executa a operação.
5. Converte o resultado em objetos JavaScript.
6. Fecha (ou reutiliza) a conexão.

Esse processo transparente simplifica o desenvolvimento e reduz significativamente os erros relacionados à persistência de dados.

1.5 Benefícios do uso de ORM

O uso de ORMs em aplicações Node.js traz diversas vantagens reconhecidas por autores e pela prática profissional (FOWLER, 2002; MARTINS; PEREIRA, 2022):

1. **Produtividade:** elimina a necessidade de escrever SQL repetitivo, acelerando o desenvolvimento.
2. **Legibilidade:** o código torna-se mais compreensível e padronizado.
3. **Portabilidade:** facilita a migração entre diferentes bancos de dados.
4. **Segurança:** reduz a exposição a vulnerabilidades, como injeções SQL.

5. **Manutenibilidade:** centraliza a lógica de acesso a dados, facilitando ajustes futuros.

Segundo a documentação oficial do Prisma (PRISMA, 2024), outra vantagem significativa é o suporte a **migrations**, que permitem controlar as mudanças de estrutura no banco de forma automatizada e versionada.

1.6 Desvantagens do ORM

O uso de ORMs em aplicações Node.js pode trazer algumas desvantagens:

- **Performance:** pode gerar consultas menos otimizadas.
- **Curva de aprendizado:** é preciso compreender a ferramenta.
- **Menor controle:** em consultas complexas, pode ser necessário SQL puro.
- **Sobrecarga:** adiciona uma camada de abstração que consome recursos.

De acordo com Fowler (2002), o uso de ORM é ideal para aplicações com **regras de negócio complexas e múltiplas entidades**, enquanto o SQL manual é mais indicado para **sistemas críticos de desempenho ou relatórios altamente específicos**.

1.7 Boas práticas de integração entre as abordagens

Muitos projetos combinam as duas estratégias — utilizam **ORM para operações padrão (CRUD)** e **SQL manual para consultas específicas**.

Por exemplo:

- CRUD de usuários, produtos e pedidos → ORM.
- Relatórios agregados e dashboards → SQL manual otimizado.

O próprio Prisma permite executar comandos SQL puros, quando necessário:

```
const resultado = await prisma.$queryRaw`SELECT COUNT(*) as total FROM users`;
```

Essa flexibilidade é o que Martins e Pereira (2022) chamam de “*abordagem híbrida*”, em que o desenvolvedor usufrui da produtividade do ORM sem abrir mão do controle total quando necessário.

2. Instalação e configuração do Prisma

2.1 O que é o Prisma ORM

O **Prisma** é um ORM moderno e de código aberto voltado para o ecossistema **Node.js**. Ele facilita a comunicação entre aplicações JavaScript/TypeScript e bancos de dados relacionais (como **MySQL**, **PostgreSQL** e **SQLite**) ou não relacionais (como **MongoDB**).

Segundo a própria documentação oficial, o Prisma foi projetado para oferecer **produtividade**, **segurança de tipos** e **clareza de código** (PRISMA, 2024).

Diferente de ORMs mais antigos, como o Sequelize, o Prisma utiliza um **arquivo de esquema declarativo** (**schema.prisma**) para definir as tabelas, colunas e relacionamentos. Esse arquivo serve como a “fonte de verdade” do banco de dados — ou seja, toda a estrutura é descrita de forma centralizada e versionável (WENDERLICH, 2021).

2.2 Etapas de instalação e configuração

A configuração do Prisma envolve algumas etapas simples que podem ser executadas diretamente no terminal do projeto Node.js.

2.2.1 Criar um projeto Node.js

Primeiro, cria-se a pasta do projeto e inicializa-se o Node.

```
mkdir prisma-intro  
cd prisma-intro
```

Em seguida, inicie o projeto criando o arquivo package.json.

```
npm init -y
```

Instale o **dotenv** para podermos configurar as variáveis de ambiente do projeto:

```
npm i dotenv
```

Você pode, agora, **abrir a aplicação no editor Visual Studio Code**.

```
code .
```

2.2.2 Instalar as dependências

Em seguida, vamos instalar o Prisma CLI e o cliente de banco de dados desejado. Neste exemplo, será utilizado **MySQL**, mas o procedimento é semelhante para outros bancos.

```
npm i -D prisma  
npm i @prisma/client
```

Neste exemplo:

- O pacote **prisma** fornece a **ferramenta de linha de comando (CLI)** para gerenciar o ORM: criar o schema, gerar o cliente e executar migrações no banco de dados.
- O pacote **@prisma/client** é o **cliente gerado automaticamente** pelo Prisma, que permite executar consultas no banco por meio de código JavaScript.

De acordo com Martins e Pereira (2022), o Prisma “automatiza a camada de persistência de dados, substituindo a escrita manual de SQL por uma interface de objetos mais expressiva e segura”.

2.2.3 Inicializar o Prisma

Após a instalação, inicialize o Prisma dentro do projeto:

```
npx prisma init
```

Esse comando cria automaticamente a seguinte estrutura de diretórios:

```
prisma-intro/  
├── prisma/  
│   └── schema.prisma    // Arquivo de definição do banco de dados  
├── .env                 // Arquivo de variáveis de ambiente  
└── prisma.config.ts     // Arquivo de configuração do CLI do Prisma
```

O arquivo **.env** é usado para armazenar informações sensíveis, como o endereço e as credenciais do banco. O arquivo **schema.prisma** é onde você descreve a estrutura do banco de dados e as regras de conexão. Já o **prisma.config.ts** é um arquivo de **configuração do CLI do Prisma**, onde você define opções como: local do arquivo **schema.prisma**, diretório de migrações, views, queries TypedSQL, e outras propriedades de comportamento da CLI.

2.2.4 Configurar a conexão com o banco de dados

Abra o arquivo **.env** e defina a variável de ambiente **DATABASE_URL**, conforme o banco de dados escolhido. Neste exemplo, usamos o MySQL:

```
DATABASE_URL="mysql://root:1234@localhost:3306/empresa"
```

O Prisma utilizará essa URL para conectar-se ao banco de dados. Veja abaixo o que cada parte da string representa: nome de usuário, senha, endereço do servidor, porta padrão do MySQL e nome do banco.

2.2.5 Definir o modelo (schema.prisma)

O arquivo **schema.prisma** descreve a **estrutura do banco de dados** e as **regras de conexão**. É um arquivo de texto (escrito em uma *Domain-Specific Language*, ou DSL) que contém três blocos principais:

```
generator  
datasource  
model
```

Altere o arquivo **schema.prisma** com a seguinte configuração:

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "mysql"  
  url = env("DATABASE_URL")  
}
```

```
// Definição de um modelo simples
model User {
  id Int @id @default(autoincrement())
  nome String
  email String @unique
}
```

Neste exemplo:

- **generator**: informa ao Prisma que será utilizado o cliente JavaScript.
- **datasource**: define o tipo de banco e a URL de conexão.
- **model**: representa uma tabela do banco; cada campo é convertido em uma coluna.

Segundo Fowler (2002), “o modelo de dados deve ser uma abstração clara do domínio da aplicação, e não um espelho exato do banco físico”. Essa é justamente a proposta do Prisma: descrever o domínio de forma legível e independente da sintaxe SQL.

2.2.6 Configurar o arquivo prisma.config.ts

Trata-se de um arquivo de **configuração do CLI do Prisma**, onde você define opções como: local do arquivo **schema.prisma**, diretório de migrações, views, queries TypedSQL, e outras propriedades de comportamento da CLI.

Logo, altere o arquivo **prisma.config.ts**:

```
// Carrega variáveis do .env (DATABASE_URL, etc.)
import "dotenv/config";
// Importa helpers tipados do novo sistema de config do Prisma
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  // Caminho do seu schema
  schema: "prisma/schema.prisma",
  // Onde as migrações ficarão gravadas
  migrations: {
    path: "prisma/migrations",
  },
});
```

```
// Usa o mecanismo padrão de execução de consultas, baseado em Rust.  
engine: "classic",  
// Define a(s) origem(ns) de dados. Aqui pegamos a URL do .env  
datasource: {  
  url: env("DATABASE_URL"),  
},  
});
```

Neste arquivo você especifica propriedades como:

- **schema**: localização do(s) arquivo(s) **schema.prisma**.
- **migrations.path**: onde ficam os scripts de migração.
- **engine: "classic"**: Usa o mecanismo padrão de execução de consultas, baseado em Rust.
- **datasource**: informações de conexão com o banco de dados.

Depois, ao executar comandos do Prisma CLI (por exemplo **npx prisma migrate dev**), o Prisma verifica esse arquivo de configuração para saber “onde” está o **schema**, onde gerar migrações, etc.

2.2.6 Migrar e gerar client

Agora, vamos indicar para o Prisma executar o **processo de migração** do banco de dados — ou seja, o Prisma compara o que está definido no arquivo **schema.prisma** com o estado atual do banco e cria/atualiza as tabelas automaticamente.

Assim, execute o comando:

```
npx prisma migrate dev --name init
```

Esse comando faz duas coisas:

1. É gerado um arquivo de migração na pasta **prisma/migrations/**;
2. O banco de dados é atualizado para refletir as mudanças do schema;
3. O cliente Prisma (**@prisma/client**) é regenerado, garantindo que o código e o banco fiquem sincronizados.

O Prisma exibirá uma mensagem indicando que a migração foi aplicada com sucesso, e o

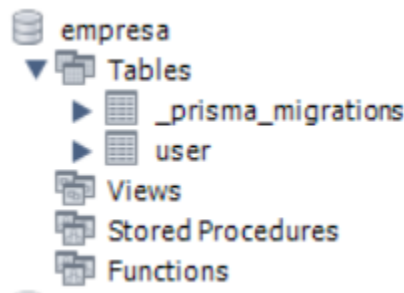
banco já estará sincronizado.

Veja abaixo o arquivo **migration.sql** criado neste comando na pasta **prisma/migrations::**

```
-- CreateTable
CREATE TABLE `User` (
  `id` INTEGER NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(191) NOT NULL,
  `email` VARCHAR(191) NOT NULL,

  UNIQUE INDEX `User_email_key` (`email`),
  PRIMARY KEY (`id`)
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Após, observe seu banco de dados MySQL:



OBS: Sempre que **mudar o schema**, rode uma **nova** migration:

```
npx prisma migrate dev --name <nome-da-mudanca>
```

2.2.7 Testar a conexão com o banco

Agora, podemos criar na raiz do projeto um arquivo **index.js** para validar a conexão:

```
// Importa o PrismaClient, gerado automaticamente pelo Prisma via schema
const { PrismaClient } = require("@prisma/client");
```

```
// Cria uma instância única do cliente para acessar o banco de dados
const prisma = new PrismaClient();

// Função principal assíncrona que executa consultas no banco
async function main() {
  // Busca todos os registros da tabela "user"
  const users = await prisma.user.findMany();

  // Exibe o resultado no console
  console.log(users);
}

// Executa a função principal
main()
  // Captura e mostra possíveis erros durante a execução
  .catch((e) => console.error(e))
  // Encerra a conexão com o banco, garantindo que os recursos sejam liberados
  .finally(async () => await prisma.$disconnect());
```

Observe neste exemplo que:

- **PrismaClient** é a classe que faz a ponte com o banco.
- O método **findMany()** executa um **SELECT * FROM user**.
- **\$disconnect()** encerra a conexão, evitando vazamentos de recursos.

Agora, execute o comando abaixo:

```
node index.js
```

Observe que o console exibirá uma lista (vazia ou com dados) dos usuários da tabela.

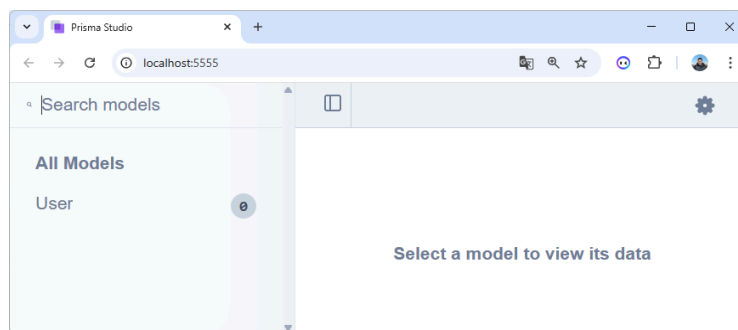
```
PS C:\Users\Mauri\OneDrive\Documentos\prisma-intro> node index.js
[]
```

2.2.8 Visualizar o banco com Prisma Studio

O Prisma oferece uma ferramenta visual chamada **Prisma Studio**, que funciona como uma interface gráfica para visualizar e editar dados. Para visualizarmos isso, execute:

```
npx prisma studio
```

Isso abrirá uma página no navegador (<http://localhost:5555>) mostrando as tabelas e registros do banco.



De acordo com Powell (2020), o Prisma Studio é uma das inovações que tornam o Prisma “não apenas um ORM, mas uma plataforma completa de gerenciamento de dados”.

Para parar sua pressione no terminal: **CTRL + C**

3 Operações CRUD com Prisma Client

Para compreender como o ORM Prisma facilita o acesso e a manipulação de dados, vamos começar com um **exemplo prático** de operações CRUD (*Create, Read, Update e Delete*). Essas operações são fundamentais em qualquer aplicação que interaja com um banco de dados, pois representam as ações básicas de inserção, leitura, atualização e remoção de informações (Fowler, 2003).

3.1 Definindo o Modelo

O modelo utilizado neste exemplo é propositalmente simples, definido no arquivo **prisma/schema.prisma**. Ele representa uma tabela chamada **User**, com três campos: **id**, **nome** e **email**. Verifique se o seu modelo está assim:

```
generator client {  
  provider = "prisma-client-js"  
}
```



```
datasource db {  
  provider = "mysql"  
  url = env("DATABASE_URL")  
}  
  
// Definição de um modelo simples  
model User {  
  id Int @id @default(autoincrement())  
  nome String  
  email String @unique  
}
```

Esse modelo indica que o campo **id** será a chave primária e será incrementado automaticamente a cada novo registro. O campo **email** é único, o que significa que não poderão existir dois usuários com o mesmo endereço de e-mail. Após salvar o arquivo, o Prisma é capaz de gerar automaticamente um cliente que permitirá manipular os dados dessa tabela por meio de JavaScript.

3.2 Criando o Prisma Client

O primeiro passo é criar uma instância do Prisma Client. Isso é feito importando a classe **PrismaClient** do pacote **@prisma/client** e inicializando-a. Em projetos maiores, é uma boa prática criar um arquivo separado (por exemplo, **prismaClient.js**) para centralizar essa configuração.

Assim, crie na raiz do projeto um arquivo chamado **prismaClient.js** com este código:

```
const { PrismaClient } = require("@prisma/client");  
const prisma = new PrismaClient();  
  
module.exports = prisma;
```

Com isso, qualquer parte do sistema pode importar o **prisma** e realizar operações no banco de dados sem precisar reconfigurar a conexão.

3.3 CREATE – Inserindo um novo registro

A primeira operação é a criação de um novo usuário. No Prisma, isso é feito com o método **create()**. Ele recebe um objeto **data** contendo os valores a serem inseridos.

Atualize o arquivo **index.js** com o seguinte código:

```
// Importa a instância única do Prisma Client configurada em prismaClient.js
const prisma = require("./prismaClient");

// Função assíncrona responsável por criar um novo registro na tabela "user"
async function criarUsuario() {
  const novoUsuario = await prisma.user.create({
    data: {
      nome: "Ana Souza",           // Campo "nome" do novo usuário
      email: "ana@ifrs.edu.br"     // Campo "email" do novo usuário
    }
  });
  console.log("Usuário criado:", novoUsuario); // Exibe o resultado da criação
}

// Função principal que executa o fluxo da aplicação
async function main() {
  try {
    criarUsuario(); // Chama a função que insere o usuário no banco
  } catch (error) {
    console.error("Erro durante a operação:", error.message); // Exibe erros
  } finally {
    await prisma.$disconnect(); // Encerra a conexão com o banco
  }
}

// Executa a função principal
main();
```

Esse comando é equivalente a uma instrução SQL como:

```
INSERT INTO User (nome, email) VALUES ('Ana Souza', 'ana@ifrs.edu.br');
```

Execute o comando no terminal:

```
node index.js
```

Observe que o Prisma retorna o registro completo após a inserção, incluindo o **id** gerado automaticamente.

```
PS C:\Users\Mauri\OneDrive\Documentos\prisma-intro> node index.js
Usuário criado: { id: 1, nome: 'Ana Souza', email: 'ana@ifrs.edu.br' }
```

Verifique em seu banco de dados que este registro foi adicionado à tabela **User**:

Result Grid			
Filter Rows:			
	id	nome	email
▶	1	Ana Souza	ana@ifrs.edu.br
✱	NULL	NULL	NULL

OBS: Adicione mais 2 usuários para facilitar a visualização das próximas funcionalidades.

3.4 READ – Consultando dados

Para ler informações do banco, o Prisma oferece vários métodos, sendo os mais comuns **findMany()** e **findUnique()**.

O método **findMany()** retorna uma lista de registros (semelhante a **SELECT * FROM User**). Assim, altere o arquivo **index.js** para criar a função **listarUsuarios()** e chamá-la no **main()**:

```
...

// Função assíncrona para listar todos os usuários do banco
async function listarUsuarios() {
  const usuarios = await prisma.user.findMany(); // Busca todos os registros da
  tabela "user"
  console.log("Lista de usuários:", usuarios); // Exibe a lista no console
}

// Função principal que executa o fluxo da aplicação
async function main() {
  try {
    listarUsuarios(); // Chama a função que busca todos usuários no banco
  } catch (error) {
    console.error("Erro durante a operação:", error.message); // Captura e
    exibe erros
  } finally {
    await prisma.$disconnect(); // Encerra a conexão com o banco
  }
}
// Executa a função principal
main();
```

Execute o comando no terminal:

```
node index.js
```

E veja o resultado no terminal:

```
PS C:\Users\Mauri\OneDrive\Documentos\prisma-intro> node index.js
Lista de usuários: [
  { id: 1, nome: 'Ana Souza', email: 'ana@ifrs.edu.br' },
  { id: 2, nome: 'João Silva', email: 'joao@ifrs.edu.br' },
  { id: 3, nome: 'Mário Leite', email: 'mario@ifrs.edu.br' }
]
```

Já o método **findUnique()** busca um único registro com base em um campo único (como **id** ou **email**). Assim, altere o arquivo **index.js** para criar a função **buscarUsuarioPorEmail()** e chamá-la no **main()**:

```
...
// Função assíncrona para buscar um usuário específico pelo e-mail
async function buscarUsuarioPorEmail() {
  const usuario = await prisma.user.findUnique({
    where: { email: "ana@ifrs.edu.br" } // Critério de busca (campo único)
  });
  console.log("Usuário encontrado:", usuario); // Exibe o resultado no console
}
...
```

Além disso, o Prisma permite aplicar filtros, ordenação e paginação com grande facilidade. Assim, altere o arquivo **index.js** para criar a função **listarUsuariosOrdenados()** e chamá-la no **main()**:

```
...
// Função assíncrona para listar os 5 primeiros usuários em ordem alfabética
async function listarUsuariosOrdenados() {
  const usuariosOrdenados = await prisma.user.findMany({
    orderBy: { nome: "asc" }, // Ordena pelo campo "nome" em ordem crescente
    take: 5 // Limita o resultado aos 5 primeiros registros
  });
  console.log("Primeiros 5 usuários ordenados por nome:", usuariosOrdenados);
}
...
```

3.5 UPDATE – Atualizando registros existentes

Para atualizar um registro, utiliza-se o método **update()**, que exige um **where** (critério de busca) e um **data** (dados a serem modificados). Assim, altere o arquivo **index.js** para criar a função **atualizarUsuarioPorEmail()** e chamá-la no **main()**:

```
...
// Função assíncrona para atualizar um usuário existente pelo e-mail
async function atualizarUsuarioPorEmail() {
  const usuarioAtualizado = await prisma.user.update({
    where: { email: "ana@ifrs.edu.br" }, // Localiza o usuário pelo e-mail
    data: { nome: "Ana Paula Souza" }   // Define o novo valor para o "nome"
  });

  // Exibe no console o resultado da atualização
  console.log("Usuário atualizado:", usuarioAtualizado);
}
...
```

Internamente, o Prisma traduz isso para:

```
UPDATE User SET nome = 'Ana Paula Souza' WHERE email = 'ana@ifrs.edu.br';
```

Porém, se o registro não existir, o Prisma lançará um erro. Por isso, é comum verificar a existência antes de atualizar, usando **findUnique()**. Assim, altere o arquivo **index.js**:

```
...
// Função assíncrona para atualizar um usuário, verificando se ele existe antes
async function atualizarUsuarioPorEmail() {
  // Primeiro, verifica se o usuário existe no banco de dados
  const usuarioExistente = await prisma.user.findUnique({
    where: { email: "ana@ifrs.edu.br" } // Busca pelo e-mail informado
  });

  if (!usuarioExistente) {
    console.log("Usuário não encontrado. Nenhuma atualização realizada.");
    return; // Encerra a função se o registro não existir
  }
}
```

```
// Atualiza o registro existente
const usuarioAtualizado = await prisma.user.update({
  where: { email: "ana@ifrs.edu.br" },
  data: { nome: "Ana Paula Souza" }
});

console.log("Usuário atualizado:", usuarioAtualizado);
}
...
```

A função **atualizarUsuarioPorEmail()** primeiro usa **findUnique()** para verificar se há um registro com o e-mail informado. Caso não exista, a execução é interrompida de forma segura. Se o registro for encontrado, o método **update()** é chamado para modificar o campo **nome**.

3.6 DELETE – Excluindo registros

A exclusão de um registro é realizada com o método **delete()**. Assim como no caso da atualização, é necessário indicar o campo de busca:

```
...
// Função assíncrona para remover um usuário pelo e-mail
async function removerUsuarioPorEmail() {
  const usuarioRemovido = await prisma.user.delete({
    where: { email: "ana@ifrs.edu.br" } // Localiza o usuário pelo "email"
  });
  console.log("Usuário excluído:", usuarioRemovido); // Exibe o registro removido no console
}
...
```

O comando SQL equivalente seria:

```
DELETE FROM User WHERE email = 'ana@ifrs.edu.br';
```

Para evitar erros, é recomendável confirmar a existência do registro antes de excluí-lo. Assim, altere o **index.js**:

```
...
// Função assíncrona para remover um usuário, confirmando antes se ele existe
async function removerUsuarioPorEmail() {
```

```
// Verifica se o usuário existe no banco de dados
const usuarioExistente = await prisma.user.findUnique({
  where: { email: "ana@ifrs.edu.br" } // Busca o usuário pelo campo "email"
});

// Se não existir, exibe uma mensagem e encerra a função
if (!usuarioExistente) {
  console.log("Usuário não encontrado. Nenhuma exclusão realizada.");
  return;
}

// Se existir, realiza a exclusão
const usuarioRemovido = await prisma.user.delete({
  where: { email: "ana@ifrs.edu.br" }
});

console.log("Usuário excluído:", usuarioRemovido);
}
...
```

4. Atualização de modelos, migrações e seed de dados

4.1 Modelagem no schema.prisma

Vimos que a modelagem é realizada em uma **DSL (Domain Specific Language)** simples e expressiva, na qual **cada modelo corresponde a uma tabela** e **cada campo representa uma coluna** do banco de dados. As **anotações** permitem definir **chaves primárias, valores padrão, índices e relações entre tabelas**, tornando o processo de estruturação dos dados mais claro e declarativo (FOWLER, 2002; WENDERLICH, 2021).

Vamos atualizar nosso modelo:

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "mysql"  
  url      = env("DATABASE_URL")  
}  
  
// 1:N (User tem muitos Post)  
model User {  
  id    Int    @id @default(autoincrement())  
  nome  String  
  email String  @unique  
  criadoEm DateTime @default(now())  
  posts Post[] // relação 1:N  
  profile Profile? // relação 1:1 com Profile (opcional)  
}  
  
// 1:N (um User pode ter vários Posts)  
model Post {  
  id    Int    @id @default(autoincrement())  
  titulo String  
  conteudo String  
  publicado Boolean @default(false)  
  categorias Categoria[] // relação N:N  
  userId Int  
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```



```
// índice para consultas por título
@@index([titulo])
}
// 1:1 (um User tem um Profile)
model Profile {
  id Int @id @default(autoincrement())
  bio String?
  userId Int @unique // garante 1:1
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

// N:N (vários posts podem ter várias categorias, e vice-versa)
model Categoria {
  id Int @id @default(autoincrement())
  nome String @unique
  posts Post []
}
```

Observe que:

- **@id**, **@default(autoincrement())**, **@unique** e **@@index** controlam PK, autoincremento e índices.
- **@relation(fields: [...], references: [...])** define **FK** e **ação referencial (onDelete/onUpdate)**.
- **Profile.userId @unique** garante **1:1**; **Post.userId** sem **@unique** implementa **1:N** (PRISMA, 2024).

Boas práticas de domínio: modele primeiro entidades e relações em termos de negócio; só depois detalhe restrições (FOWLER, 2002).

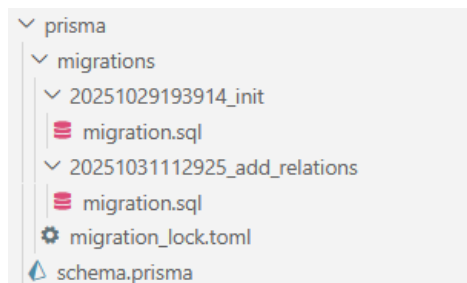
4.2 Migrações: criar, versionar e aplicar

Migrações são scripts versionados (pasta **prisma/migrations/**) que descrevem a **evolução do esquema físico** ao longo do tempo. Diferem de “rodar SQL solto” porque criam um **histórico rastreável** (PRISMA, 2024; POWELL, 2020).

Sempre que alterar modelos, crie outra migração:

```
npx prisma migrate dev --name add-relations
```

Observe que, dentro da pasta **prisma/migrations/** é criada uma nova pasta com nome similar a **20251031112925_add_relations**.



Nesta pasta, deve ter um arquivo **migration.sql** criado neste comando. Inspecione o SQL gerado para vermos **o que o ORM faz “por baixo”** (MARTINS; PEREIRA, 2022).

```
-- AlterTable
ALTER TABLE `user` ADD COLUMN `criadoEm` DATETIME(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP(3);

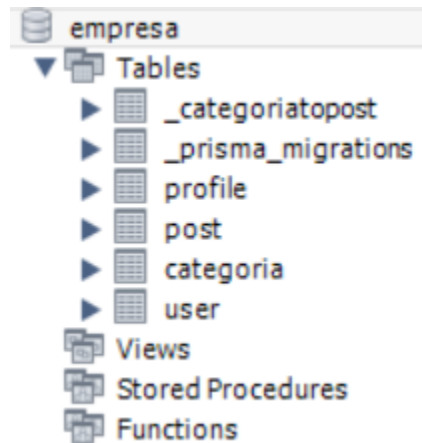
-- CreateTable
CREATE TABLE `Post` (
  `id` INTEGER NOT NULL AUTO_INCREMENT,
  `titulo` VARCHAR(191) NOT NULL,
  `conteudo` VARCHAR(191) NOT NULL,
  `publicado` BOOLEAN NOT NULL DEFAULT false,
  `userId` INTEGER NOT NULL,

  INDEX `Post_titulo_idx` (`titulo`),
  PRIMARY KEY (`id`)
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- CreateTable
CREATE TABLE `Profile` (
  `id` INTEGER NOT NULL AUTO_INCREMENT,
```

```
`bio` VARCHAR(191) NULL,  
`userId` INTEGER NOT NULL,  
  
    UNIQUE INDEX `Profile_userId_key` (`userId`),  
    PRIMARY KEY (`id`)  
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
-- CreateTable  
CREATE TABLE `Categoria` (  
    `id` INTEGER NOT NULL AUTO_INCREMENT,  
    `nome` VARCHAR(191) NOT NULL,  
  
    UNIQUE INDEX `Categoria_nome_key` (`nome`),  
    PRIMARY KEY (`id`)  
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
-- CreateTable  
CREATE TABLE `_CategoriaToPost` (  
    `A` INTEGER NOT NULL,  
    `B` INTEGER NOT NULL,  
  
    UNIQUE INDEX `_CategoriaToPost_AB_unique` (`A`, `B`),  
    INDEX `_CategoriaToPost_B_index` (`B`)  
) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
-- AddForeignKey  
ALTER TABLE `Post` ADD CONSTRAINT `Post_userId_fkey` FOREIGN KEY (`userId`)  
REFERENCES `User` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;  
  
-- AddForeignKey  
ALTER TABLE `Profile` ADD CONSTRAINT `Profile_userId_fkey` FOREIGN KEY  
(`userId`) REFERENCES `User` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;  
  
-- AddForeignKey  
ALTER TABLE `_CategoriaToPost` ADD CONSTRAINT `_CategoriaToPost_A_fkey` FOREIGN  
KEY (`A`) REFERENCES `Categoria` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;  
  
-- AddForeignKey  
ALTER TABLE `_CategoriaToPost` ADD CONSTRAINT `_CategoriaToPost_B_fkey` FOREIGN  
KEY (`B`) REFERENCES `Post` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

Observe, novamente, as alterações em seu banco de dados MySQL:



4.3 Seed de dados

O termo *seed* vem de “semente”, ou seja, **dados iniciais** para povoar o banco de dados. Serve para criar **usuários, posts, perfis, produtos, categorias, etc.**, de forma automatizada.

No Prisma, o seed é apenas um **script** que:

1. Conecta-se ao banco usando o **PrismaClient**;
2. Executa comandos como **create()** ou **upsert()** para inserir registros;
3. Encerra a conexão ao final.

Desta forma, crie um arquivo **prisma/seed.js** com o seguinte código:

```
/* Seed para: User (1:N Post, 1:1 Profile) e Categoria (N:N Post) */
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

async function main() {
  console.log("Limpendo dados anteriores...");
  // Limpa dados (ordem para evitar erros de FKs)

  await prisma.post.deleteMany();
  await prisma.profile.deleteMany();
  await prisma.categoria.deleteMany();
  await prisma.user.deleteMany();

  console.log("Inserindo categorias...");
```

```
// Categorias base (N:N serão conectadas pelo nome)
await prisma.categoria.createMany({
  data: [
    { nome: "Node.js" },
    { nome: "Educação" },
    { nome: "Backend" },
    { nome: "DevOps" },
  ],
  skipDuplicates: true,
});

console.log("Criando usuários, perfis e posts...");
// Usuário 1 + Profile + Posts (com categorias conectadas pelo nome)
const ana = await prisma.user.create({
  data: {
    nome: "Ana Souza",
    email: "ana@ifrs.edu.br",
    profile: { create: { bio: "Professora de Desenvolvimento Web" } },
    posts: {
      create: [
        {
          titulo: "ORM com Prisma",
          conteudo: "Modelos, migrações e relações.",
          publicado: true,
          categorias: { connect: [{ nome: "Node.js" }, { nome: "Backend" }] },
        },
        {
          titulo: "Migrações na prática",
          conteudo: "Como versionar seu banco.",
          publicado: false,
          categorias: { connect: [{ nome: "DevOps" }] },
        },
      ],
    },
  },
  include: { profile: true, posts: { include: { categorias: true } } },
});

console.log("Usuário criado:", ana);

// Usuário 2 (exemplo minimalista)
const bruno = await prisma.user.create({
  data: {
    nome: "Bruno Lima",
    email: "bruno.lima@ifrs.edu.br",
```

```
profile: { create: { bio: "Engenheiro de Software" } },
posts: {
  create: [
    {
      titulo: "Índices e performance",
      conteudo: "Quando e por que usar.",
      publicado: true,
      categorias: { connect: [{ nome: "Backend" }] },
    },
  ],
},
include: { profile: true, posts: { include: { categorias: true } } },
});
console.log("Usuário criado:", bruno);

console.log("Seed concluído.");
}

main()
  .catch((e) => {
    console.error("Erro no seed:", e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
```

Este código começa **limpando as tabelas (post, profile, categoria, user)** na ordem correta para evitar erros de chave estrangeira. Em seguida, **cria categorias básicas** que servirão como referência para os posts. Depois, insere **usuários completos**, com seus respectivos **perfis (relação 1:1)** e **posts (relação 1:N)**, conectando cada post às categorias existentes por meio de uma **relação N:N**. O script usa **include** para retornar no console todos os dados relacionados, permitindo verificar se as associações foram criadas corretamente. Por fim, o código encerra a conexão com o banco, garantindo que os recursos sejam liberados após a execução.

Em seguida, atualize o arquivo **prisma.config.ts** para indicar o comando seed:

```
// Carrega variáveis do .env (DATABASE_URL, etc.)
import "dotenv/config";
```

```
// Importa helpers tipados do novo sistema de config do Prisma
import { defineConfig, env } from "prisma/config";
export default defineConfig({
  // Caminho do seu schema
  schema: "prisma/schema.prisma",
  // Onde as migrações ficarão gravadas
  migrations: {
    path: "prisma/migrations",
    seed: "node prisma/seed.js",
  },
  // Usa o mecanismo padrão de execução de consultas, baseado em Rust.
  engine: "classic",
  // Define a(s) origem(ns) de dados. Aqui pegamos a URL do .env
  datasource: {
    url: env("DATABASE_URL"),
  },
});
```

Em seguida, vamos executar o seed:

```
npx prisma db seed
```

Veja o resultado desta execução no terminal

```
Limpando dados anteriores...
Inserindo categorias...
Criando usuários, perfis e posts...
Usuário criado: {
  id: 4,
  nome: 'Ana Souza',
  email: 'ana@ifrs.edu.br',
  criadoEm: 2025-10-31T11:55:51.240Z,
  profile: { id: 1, bio: 'Professora de Desenvolvimento Web', userId: 4 },
  posts: [
    {
      id: 1,
      titulo: 'ORM com Prisma',
      conteudo: 'Modelos, migrações e relações.',
      publicado: true,
      userId: 4,
      categorias: [Array]
    },
    {
      id: 2,
      titulo: 'Migrações na prática',
      conteudo: 'Como versionar seu banco.',
      publicado: false,
      userId: 4,
      categorias: [Array]
    }
  ]
}
```

E, depois, observe que os dados foram populados no banco de dados. Por exemplo, observe como ficou a tabela “user” no MySQL:

Result Grid				
Filter Rows:				
Edit:   				
	id	nome	email	criadoEm
▶	4	Ana Souza	ana@ifrs.edu.br	2025-10-31 11:55:51.240
	5	Bruno Lima	bruno.lima@ifrs.edu.br	2025-10-31 11:55:51.257
*	NULL	NULL	NULL	NULL

Veja como ficou a tabela “post” no MySQL:

Programa de Residência Tecnológica em Desenvolvimento de Software: Bolsa Futuro Digital Curso: Desenvolvedor Back-End

Result Grid   Filter Rows: Edit:    Export/Import: 					
	id	titulo	conteudo	publicado	userId
▶	1	ORM com Prisma	Modelos, migrações e relações.	1	4
	2	Migrações na prática	Como versionar seu banco.	0	4
	3	Índices e performance	Quando e por que usar.	1	5
*	NULL	NULL	NULL	NULL	NULL

Veja como ficou a tabela “profile” no MySQL:

Result Grid   Filter Rows: Edit: 			
	id	bio	userId
▶	3	Professora de Desenvolvimento Web	6
	4	Engenheiro de Software	7
*	NULL	NULL	NULL

Veja como ficou a tabela “categoria” no MySQL:

Result Grid   Filter Rows:		
	id	nome
▶	7	Backend
	8	DevOps
	6	Educação
	5	Node.js
*	NULL	NULL

5 Relações (1:1, 1:N, N:N) em nível de código

Ao trabalhar com bancos de dados relacionais, as entidades (ou tabelas) raramente estão isoladas. Elas se conectam através de **relacionamentos**, que indicam como os registros se ligam uns aos outros. Essas relações podem ser de **um para um (1:1)**, **um para muitos (1:N)** ou **muitos para muitos (N:N)**. O ORM Prisma facilita esse trabalho porque permite representar essas relações **diretamente no código**, sem precisar escrever SQL manualmente.

5.1 Relação 1:1 (User ↔ Profile)

Uma relação **1:1** ocorre quando **um registro de uma tabela** se relaciona com **um único registro de outra**. Exemplo clássico: um usuário tem **um único perfil**, e cada perfil pertence a **apenas um usuário**.

Vejamos um exemplo de como criar usuário com perfil junto (inclusão aninhada)

```
// Função assíncrona para criar um usuário com seu perfil (relação 1:1)
async function criarUsuarioComProfile() {
  const novoUsuario = await prisma.user.create({
    data: {
      nome: "Diego Martins",
      email: "diego.martins@ifrs.edu.br",

      // Criação aninhada do perfil associado ao usuário (1:1)
      profile: {
        create: {
          bio: "Analista de Sistemas e entusiasta de Inteligência Artificial",
        },
      },
    },
  },

  // Retorna também os dados do perfil relacionado
  include: {
    profile: true,
  },
});

  console.log("Usuário e perfil criados:", novoUsuario);
}
```

Neste exemplo, o Prisma cria o usuário e **automaticamente associa o perfil**, cuidando da

chave estrangeira (**userId**) internamente. Sem ORM, seria necessário criar o usuário primeiro, pegar o **id** e depois inserir manualmente o perfil com esse **id**.

Veja um exemplo de como criar o perfil depois, conectando a um usuário já existente.

```
// Função assíncrona para criar um Profile vinculado a um User existente (relação 1:1)
async function criarProfileParaUsuarioExistente() {
  const perfil = await prisma.profile.create({
    data: {
      bio: "Usuária nova com perfil criado posteriormente",

      // Conecta este Profile a um User que já existe (exemplo: id = 1)
      user: {
        connect: { id: 1 },
      },
    },

    // Inclui o usuário vinculado no retorno, para visualizar a relação
    include: {
      user: true,
    },
  });
  console.log("Perfil criado e vinculado ao usuário:", perfil);
}
```

Neste código, **connect** cria a relação entre o perfil e o usuário existente. Isso é útil quando o registro pai (usuário) já foi criado antes.

Agora, veja como desconectar a relação 1:1 entre **User** e **Profile** e deletar o perfil:

```
// Função assíncrona para demonstrar como remover ou deletar a relação 1:1
async function removerOuDeletarProfile() {
  // 1. Remove a relação (desconecta o perfil do usuário, mas mantém o Profile)
  // Só funciona se o campo userId em Profile NÃO for obrigatório no schema.
  await prisma.user.update({
    where: { id: 1 }, // usuário alvo
    data: { profile: { disconnect: true } },
  });
  console.log("Relação User ↔ Profile desconectada (perfil mantido).");

  // 2. Remove completamente o registro do Profile vinculado ao usuário
}
```

```
// Essa operação deleta o perfil do banco de dados.  
await prisma.user.update({  
  where: { id: 1 },  
  data: { profile: { delete: true } },  
});  
console.log("Perfil associado ao usuário removido definitivamente.");  
}
```

Veja que usamos **disconnect** para remover a relação de perfil com user. Porém, ele mantém o registro do perfil.

5.2 Relação 1:N (User → Post)

Uma relação **1:N** ocorre quando **um registro (pai)** está ligado a **vários registros (filhos)**. Por exemplo: um usuário pode ter **vários posts**, mas cada post pertence a **apenas um usuário**.

Veja abaixo como criar um usuário com posts:

```
// Função assíncrona para demonstrar a relação 1:N (User → Post)  
async function criarUsuarioComPosts() {  
  const usuarioComPosts = await prisma.user.create({  
    data: {  
      nome: "Carlos Silva",  
      email: "carlos.silva@ifrs.edu.br",  
      // Criação aninhada de múltiplos posts para o mesmo usuário (relação 1:N)  
      posts: {  
        create: [  
          {  
            titulo: "Introdução ao Prisma",  
            conteudo: "Explorando o ORM e suas principais funcionalidades.",  
            publicado: true,  
            // Relaciona este post a categorias existentes (opcional)  
            categorias: {  
              connect: [{ nome: "Node.js" }, { nome: "Backend" }],  
            },  
          },  
          {  
            titulo: "Entendendo Relações 1:N",  
            conteudo: "Como modelar e popular dados relacionados com Prisma.",  
            publicado: false,  
            categorias: {  
              connect: [{ nome: "Educação" }],  
            },  
          },  
        ],  
      },  
    },  
  });  
}
```

```
    },  
  },  
],  
},  
},  
// Inclui no retorno os posts criados e suas categorias associadas  
include: {  
  posts: { include: { categorias: true } },  
},  
});  
console.log("Usuário criado com posts:", usuarioComPosts);  
}
```

Observe que o Prisma cria o usuário e seus posts de uma vez. O **include** faz com que o retorno mostre o usuário **e todos os posts criados**.

Agora, vejamos como criar um post e conectar ao usuário existente:

```
// Função assíncrona para criar um novo post vinculado a um usuário existente  
async function criarPostParaUsuario() {  
  const postNovo = await prisma.post.create({  
    data: {  
      titulo: "Monitoramento de APIs com Prisma e Node.js",  
      conteudo: "Aprendendo a integrar logs e métricas em projetos Express.",  
      publicado: true,  
  
      // Conecta o post ao usuário existente (exemplo: id = 2)  
      user: {  
        connect: { id: 2 },  
      },  
  
      // Opcional: conecta o post a categorias já existentes  
      categorias: {  
        connect: [{ nome: "Node.js" }, { nome: "DevOps" }],  
      },  
    },  
  
    // Inclui o usuário e as categorias associadas no retorno  
    include: {  
      user: true,  
      categorias: true,  
    },  
  });  
}
```

```
console.log("Post criado e vinculado ao usuário:", postNovo);  
}
```

Observe que o **connect** é útil para inserir filhos depois, sem precisar repetir os dados do pai. O Prisma cuida automaticamente do **userId** na tabela **Post**.

Agora, veja como realizar consultas com **include** e **select**:

```
// Função assíncrona para demonstrar consultas com include e select  
async function buscarUsuarioComPosts() {  
  // include: retorna o objeto completo com todos os posts do usuário  
  const userComPosts = await prisma.user.findUnique({  
    where: { id: 2 }, // usuário existente  
    include: {  
      posts: {  
        include: { categorias: true }, // inclui também as categorias dos posts  
      },  
      profile: true, // opcional: traz o perfil (relação 1:1)  
    },  
  });  
  // select: retorna apenas campos específicos do usuário e dos posts  
  const userSlim = await prisma.user.findUnique({  
    where: { id: 2 },  
    select: {  
      nome: true,  
      email: true,  
      posts: {  
        select: {  
          titulo: true,  
          publicado: true,  
          categorias: { select: { nome: true } }, // mostra apenas o nome das  
categorias  
        },  
      },  
    },  
  });  
}
```

```
    },  
  },  
});  
console.log("Usuário completo com posts e categorias:", userComPosts);  
console.log("Usuário resumido (campos selecionados):", userSlim);  
}
```

Observe que usamos **include** para retornar os dados completos do relacionamento (útil para debug). Usamos o **select** para definir quais campos serão retornados.

5.3 Relação N:N (Post ↔ Categoria)

Uma relação **N:N (muitos para muitos)** ocorre quando vários posts podem ter várias categorias, e vice-versa. O Prisma cria automaticamente uma **tabela de junção** para representar essas ligações.

Veja um exemplo de como criar post e conectar categorias:

```
// Função assíncrona para criar um post com categorias associadas (relação N:N)  
async function criarPostComCategorias() {  
  const postComCategorias = await prisma.post.create({  
    data: {  
      titulo: "Usando Prisma com Node.js",  
      conteudo: "Explorando a relação N:N entre Post e Categoria.",  
      publicado: true,  
  
      // Conecta o post a um usuário existente (relação N:1 → User)  
      user: {  
        connect: { id: 1 }, // exemplo: usuário com id = 1  
      },  
  
      // Cria categorias novas ou conecta às já existentes (relação N:N)  
      categorias: {  
        connectOrCreate: [  
          {  
            where: { nome: "Node.js" },  
            create: { nome: "Node.js" },  

```

```
    },  
    {  
      where: { nome: "ORM" },  
      create: { nome: "ORM" },  
    },  
  ],  
},  
},  
// Inclui no retorno as categorias associadas e o usuário  
include: {  
  categorias: true,  
  user: true,  
},  
});  
console.log("Post criado com categorias e usuário associado:",  
postComCategorias);  
}
```

O **connectOrCreate** evita duplicar categorias já existentes, garantindo integridade. Isso é muito útil quando o mesmo termo pode ser reutilizado (como "Node.js").

Vejamos como desconectar uma categoria:

```
// Função assíncrona para remover o vínculo N:N entre um Post e uma Categoria  
async function removerVinculoPostCategoria() {  
  // Remove a associação entre o post (id = 3) e a categoria (id = 2),  
  // mas mantém ambos os registros no banco de dados.  
  const postAtualizado = await prisma.post.update({  
    where: { id: 3 }, // Post existente  
    data: {  
      categorias: {  
        disconnect: { id: 2 }, // remove apenas o vínculo N:N  
      },  
    },  
    include: { categorias: true }, // retorna as categorias restantes após o  
    unlink  
  });  
  
  console.log("Vínculo entre post e categoria removido:", postAtualizado);  
}
```

Veja que este código remove o vínculo entre o post e a categoria, sem apagar nenhum dos dois.

Agora, vejamos como consultar posts e categorias:

```
// Função assíncrona para listar posts com autor e categorias associadas
async function listarPostsComRelacionamentos() {
  const posts = await prisma.post.findMany({
    include: {
      // Inclui o autor do post (relação N:1 → User)
      user: {
        select: { nome: true, email: true }, // mostra apenas nome e email
      },

      // Inclui todas as categorias associadas (relação N:N → Categoria)
      categorias: {
        select: { nome: true }, // mostra apenas o nome das categorias
      },
    },
  });
  console.log("Lista de posts com autor e categorias:", posts);
}
```

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- EVANS, E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2004.
- FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.
- MARTINS, João; PEREIRA, Luís. Desenvolvimento Web com Node.js e APIs RESTful. São Paulo: Novatec, 2022.
- OWASP Foundation. SQL Injection Prevention Cheat Sheet. 2023. Disponível em: <https://owasp.org>. Acesso em: 27 out. 2025.
- POWELL, Adam. Node.js Design Patterns for Enterprise Applications. New York: Packt Publishing, 2020.
- PRISMA. Prisma Documentation: Getting Started with Prisma ORM. 2024. Disponível em: <https://www.prisma.io/docs>. Acesso em: 27 out. 2025.
- SEQUELIZE CONTRIBUTORS. Sequelize Documentation. 2025. Disponível em: <https://sequelize.org/>. Acesso em: 29 out. 2025.
- TYPEORM TEAM. TypeORM Documentation. 2025. Disponível em: <https://typeorm.io/>.

**Programa de Residência Tecnológica em
Desenvolvimento de Software:
Bolsa Futuro Digital
Curso: Desenvolvedor Back-End**

Acesso em: 29 out. 2025.

OBJECTION.JS. Objection.js Guide. 2025. Disponível em:
<https://vincit.github.io/objection.js/>. Acesso em: 29 out. 2025.

- WENDE, Christian. Understanding Object-Relational Mapping (ORM). Journal of Software Engineering Practices, v. 7, n. 3, p. 12–21, 2019.
- WENDERLICH, Ray. Full Stack Development with Node.js and Prisma. New York: RW Press, 2021.